
Java Project Work in Financial Mathematics

Alessandro Gnoatto

`alessandro.gnoatto@univr.it`

March 20, 2026

Chapter 1

The starting situation

1.1 Introduction

Today is your first day at Mid-Sized-Bank PLC as a quant. Mid-Sized-Bank PLC is not a huge market player. They are making their first steps in the derivative business due to the special demands of certain clients who want to hedge risks by using simple products.

Instead of buying a software from a vendor, they decided to build the analytics internally and they recently hired a seasoned quant, your direct boss, to supervise a small and growing group of people.

1.2 The existing library of the bank

What the small group of quants has achieved so far is rather limited. They just have a basic pricing library that prices European and American options by using three different tree models:

1. Cox Ross Rubinstein
2. Boyle (trinomial)
3. Jarrow Rudd

The library is written in Java and has the structure in Figure 1.1. It is well organized in a series of packages with a clear object-oriented model in mind. Let us take a general look:

1.2.1 `net.finmath.modelling`

This is a service package where we put concepts that relate to financial products in general, like the enumeration `CallOrPut` so that it can be used in the future by other numerical methods such as Monte Carlo.

1.2.2 net.finmath.tree

This is the first package of the tree framework. At this level we do not know if we are talking about equity or interest rate models and products. Here we describe the concepts which are general to any tree model or tree product independent of the asset class of the underlying. The starting points of the hierarchies of classes are two interfaces.

1. **TreeProduct** which declares only one method

```

1      /**
2      * Prices this product under the given tree model.
3      */
4      double getValue(TreeModel model );
5  
```

2. **TreeModel** which provides some general methods that a tree product should provide. The most important are:

```

1      /**
2      * Returns the spot process at a given time transformed pointwise by a function
3      * .
4      * Equivalent to getSpotAtGivenTimeIndexRV(k).apply(transformFunction), but
5      * accepts a physical time.
6      *
7      * @param timeindex Physical time t; the model maps it to the nearest lattice
8      * index k (e.g. round(t/dt)).
9      * @param transformFunction Function f(s) applied element-wise to S_k.
10     * @return A RandomVariable at time k with length = number of states at level k
11     * (binomial: k+1; trinomial: 2k+1), containing f(S_k) pathwise.
12     */
13     RandomVariable getTransformedValuesAtGivenTimeRV(double timeindex,
14     DoubleUnaryOperator transformFunction);
15
16     /**
17     * One-step risk-neutral discounted conditional expectation operator.
18     * Takes values defined at level k+1 and projects them back to level k:
19     *  $V_k = E^Q[V_{k+1} | F_k] * DF$  (DF is the one-step discount factor).
20     *
21     * @param optionValues RandomVariable living at level k+1.
22     * @param timeindex Level k to which the conditional expectation is taken
23     * (0 < k < n).
24     * @return A RandomVariable at level k with length matching the number of
25     * states at k.
26     */
27     RandomVariable getConditionalExpectationRV(RandomVariable optionValues, int
28     timeindex);
29 
```

The first one applies a function to a random variable (this is what you need to do when you compute the payoff at each possible realization of the underlying) and returns clearly a `RandomVariable` of `finmath`. The second is also very important in view of pricing, because it computes the one step conditional expectation that you need when you price a European option or you compute the Snell envelope.

3. the next model class is `OneDimensionalRiskFactorTreeModel` Still, we are not specifying

an asset class, but we are simply saying that we are simulating only one stock or only one interest rate etc. Here some more logic is attached and a few method are introduced.

4. **AbstractTreeProduct** is interesting, because we see three method that seem to do the same thing, however:

(a) The method

```

1      public double getValue(final TreeModel model) {
2          final RandomVariable v0 = getValue(0.0, model);
3          return v0.get(0);
4      }
5  
```

computes the price only at time zero, which is a number.

(b) The method

```

1      public RandomVariable getValue(final double evaluationTime, final
TreeModel model) {
2          final RandomVariable[] levels = getValues(0.0, model);
3          // This is dangerous as it assumes a uniform time discretization.
4          final int index = (int) Math.round(evaluationTime / model.getTimeStep
());
5          return levels[index];
6      }
7  
```

computes the value of the derivative at a generic point in time, hence it returns a **RandomVariable**

(c) The final one

```

1      public abstract RandomVariable[] getValues(double evaluationTime,
TreeModel model);
2  
```

returns an array of **RandomVariable** which provides the subtree of the price from **evaluationTime** up to the maturity of the contract. In the library, we call this method by always specifying **evaluationTime** = 0.0, meaning that we compute the whole tree of the price from the beginning until the end.

1.2.3 net.finmath.tree.assetderivativevaluation

From this subpackage we will assume that we are dealing with tree methods that discretize stocks. What we do here is introduce an interface that requires the implementation of methods more closely related to a stock:

```

1      package net.finmath.tree.assetderivativevaluation;
2
3      /**
4       * From this level of abstraction, we mean that the tree should represent a stock price
5       * and not, for example, an interest rate.
6       *
7       * @author Alessandro Gnoatto
8       */

```

```

9 public interface OneDimensionalAssetTreeModel {
10
11     /**
12      * The risk factor is a stock.
13      *
14      * @return The initial stock price.
15      */
16     double getInitialPrice();
17
18     /**
19      * The risk free rate.
20      *
21      * @return The risk free rate.
22      */
23     double getRiskFreeRate();
24
25     /**
26      * Returns the volatility.
27      *
28      * @return The volatility.
29      */
30     double getVolatility();
31 }

```

1.2.4 net.finmath.tree.assetderivativevaluation.models

In this package we have exactly one class for each tree method supported by the bank:

1. CoxRossRubinsteinModel the binomial model from the lecture with the standard parameters:

$$u = e^{\sigma\sqrt{\Delta t}}, \quad d = \frac{1}{u}, \quad p = \frac{e^{r\Delta t} - d}{u - d}$$

2. BoyleTrinomial

$$\begin{aligned}
 u &= e^{\sigma\sqrt{2\Delta t}} \\
 d &= \frac{1}{u} = e^{-\sigma\sqrt{2\Delta t}} \\
 m &= 1
 \end{aligned}$$

The code defines

$$R = e^{r\Delta t}$$

and the probabilities of going up, down or stay constant are

$$\begin{aligned}
 p_u &= \frac{(R - 1)(d^2 - 1) - (M_2 - 1)(d - 1)}{(u - 1)(d^2 - 1) - (d - 1)(u^2 - 1)} \\
 p_d &= \frac{(u - 1)(M_2 - 1) - (u^2 - 1)(R - 1)}{(u - 1)(d^2 - 1) - (d - 1)(u^2 - 1)} \\
 p_m &= 1 - p_u - p_d
 \end{aligned}$$

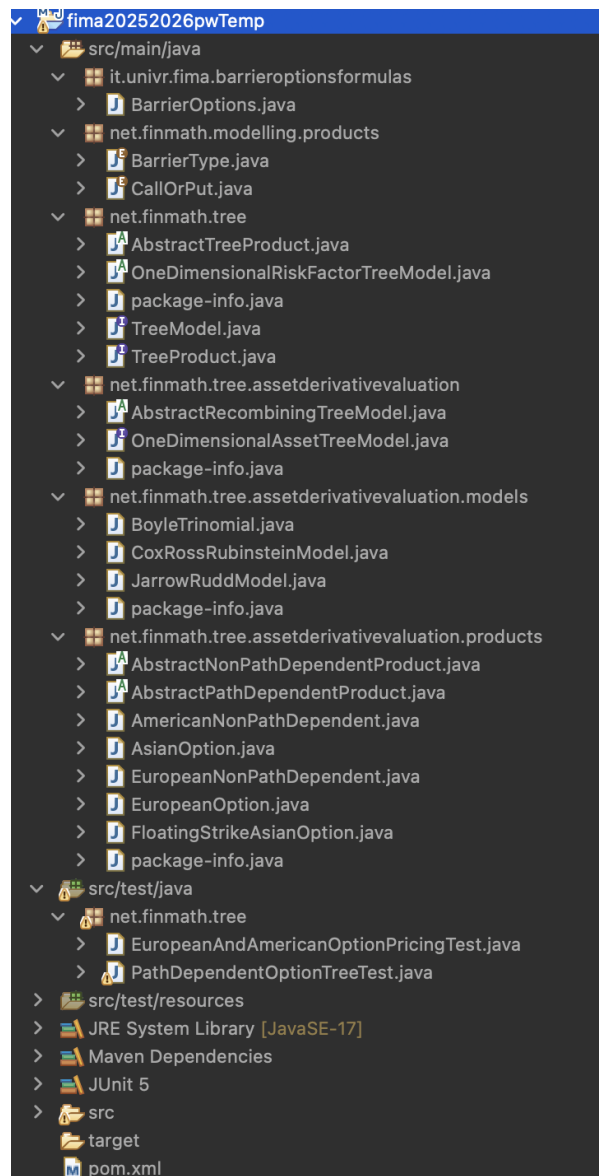


Figure 1.1: The Eclipse project template

3. `JarrowRuddModel` In this case the risk neutral probabilities of going up or down are the same: 1/2 like in a coin toss. The grow/degrowth parameters of the stock implemented in the class are

$$\begin{aligned}\mu^* &= \left(r - \frac{1}{2}\sigma^2\right) \Delta t \\ \nu &= \sigma\sqrt{\Delta t} \\ u &= e^{\mu^* + \nu} = \exp\left(\left(r - \frac{1}{2}\sigma^2\right) \Delta t + \sigma\sqrt{\Delta t}\right) \\ d &= e^{\mu^* - \nu} = \exp\left(\left(r - \frac{1}{2}\sigma^2\right) \Delta t - \sigma\sqrt{\Delta t}\right)\end{aligned}$$

Pay attention: all these formulas are implemented under the rather textbook/academic assumption that Δt is constant over the whole discretization of time. This is a clear weakness of this software design and something you should keep in mind.

1.3 `net.finmath.tree.assetderivativevaluation.products`

This is the package where we store the classes for the products. For the purpose of this project work, you can ignore/forget the following classes because we will never use them¹:

- `AbstractPathDependentProduct`
- `AsianOption`
- `FloatingStrikeAsianOption`

The relevant product classes for this project are

- `EuropeanNonPathDependent`
- `AmericanNonPathDependent`

These classes provide the price of a generic European or American option by allowing you to specify a generic `DoubleUnaryOperator` as the payoff functional. For European Call and Puts, there is also a specialized class `EuropeanOption`.

¹To implement path dependent products, we had to come up with a very complicated trick by working with array indices - the problem is that while the stock price is recombining, the path dependent functional is not as you know from the final part of chapter 1. The `AsianOption` however works and is benchmarked against a Monte Carlo simulation in the test classes. However, never use it with too many time steps in case you are curious. Non-recombining trees can quickly exhaust the memory of your computer.

Chapter 2

The exercises

It would be best to solve Exercise 2 by first solving Exercise 1. If you want to avoid solving Exercise 1 the alternative would be to hard-code dividends in the model classes: this is possible and can work, but is not very elegant. Exercises 4 and 5 can be solved instead independently from the part on dividends. This might help split the work among group members.

2.1 The first problem

So far the bank only traded very short-dated options but now clients are starting to demand long-dated products. So far the pricing library worked under the assumption that the underlying does not pay dividends, which was roughly acceptable for very short-dated options. However, for an option that matures in for example 5 years, it could well be that the firm pays dividends 5 times and then the underlying stock will jump downwards 5 times too. This introduces a bias in our prices. The boss asks you to find ways to include dividends in the pricing library.

You start looking at the literature and then you see that there are many alternatives to model dividends.

1. **ProportionalDividend** In this case the stock price behaves like:

$$\begin{aligned} S_t &= S_t^{\text{nodiv}} * D(t) \\ &= S_t^{\text{nodiv}} * (1 - \delta) \end{aligned}$$

where δ is the percentage amount of the dividend. With this model a recombining tree remains recombining, which is good. There is no impact on the calculation of the up/down or up/down/mid factors and hence the risk neutral probabilities are not affected.

2. **DiscreteProportionalDividends** This is the generalization of the previous for the case where we have multiple dividends because we write

$$D(t) = \prod_{t_k \leq t} (1 - \delta_k)$$

meaning that

$$\begin{aligned} S_t &= S_t^{\text{nodiv}} * D(t) \\ &= S_t^{\text{nodiv}} * \prod_{t_k \leq t} (1 - \delta_k) \end{aligned}$$

and again a recombining tree remains recombining. There is no impact on the calculation of the up/down or up/down/mid factors and hence the risk neutral probabilities are not affected.

3. **ContinuousDividendYield** The dividend is transformed into an artificial discount factor of the form

$$D(t) = e^{-\int_0^t q(s)ds}$$

This is nice because this is a choice that is also very common in continuous time models, however one should be careful: if the dividend yield q is not a constant, then there will be an impact on the up and down probabilities at each time steps so, if we implement it, we should warn the user and tell him/her that this is ok only as long as $q(t) \equiv q$ for q a constant dividend yield. If the dividend yield is not constant, the model becomes time-inhomogeneous and the transition probabilities must be recomputed at each time step. In this project, you may assume constant dividend yield. Again: if q is not constant then there is an impact on the calculation of the up/down or up/down/mid factors and hence the risk neutral probabilities change at every step of the tree.

4. **DiscreteCashDividends** This is the most realistic situation because you have

$$S_t = S_t^{\text{nodiv}} - D(t)$$

but this is a numerical nightmare because recombining tree models adopting such type of dividends are no longer recombining!

2.2 Exercise 1: introduce a class hierarchy for the representation of dividends

The first exercise asks you to consider the impact of dividends on option pricing models.

1. Write a `DividendModel` interface which regroups the common features of any dividend specification. If you believe you do not need this in your specification, you are free to write

```
1 public interface DividendModel {
2
3     }
4
```

an empty interface.

2. Assuming that there will be only one dividend is a receipt for a disaster, I suggest that

you view each dividend payment as a `DividendEvent` (interface) with a `double getTime()` telling you when the dividend will be paid.

3. **Hint** try to see if this interface can be useful for you, you are not forced to use it if you want to go your way:

```

1      public interface MultiplicativeDividendModel extends DividendModel {
2
3      /**
4       * Returns the cumulative multiplicative dividend factor up to the given time.
5       *
6       * @param time The evaluation time.
7       * @return The cumulative dividend factor.
8       */
9      double getCumulativeDividendFactor(double time);
10
11     /**
12      * Returns the forward multiplicative dividend factor between two times.
13      *
14      * @param time1 Start time.
15      * @param time2 End time.
16      * @return The forward dividend factor over {@code (time1,time2]}.
17      */
18     double getForwardDividendFactor(double time1, double time2);
19 }
20

```

4. At this point you should be able to write concrete classes. Try to write

- (a) `ProportionalDividend`
- (b) `DiscreteProportionalDividends`
- (c) `ContinuousDividendYield`

avoid the `DiscreteCashDividend` since it will break everything in your library.

IMPORTANT If dividend dates do not fall exactly on the time grid, map them to the first grid point greater than or equal to the dividend time for simplicity. In this project we assume a uniform time grid. Therefore, instead of refining the grid, we map dividend dates to the next grid point. Including extra time points in order to cover the dates where the dividends are paid could violate the assumption of a uniform time discretization (which affect the way risk neutral probabilities are computed!).

2.3 Exercise 2: modify the existing models to include dividends

Modify the classes for `CoxRossRubinsteinModel`, `BoyleTrinomial` and `JarrowRudd` in order to account for dividends. Of course, the case where the stock does not pay dividends should still be possible. If you want, you can write a `NoDividends` class to handle this case or but you do not need to. My suggestion is that you keep the current constructors as they are (backward compatibility) and introduce additional constructors that accept as input the dividends classes.

IMPORTANT In recombining trees, discrete proportional dividends only affect the node values and do not modify the transition probabilities. Continuous dividend yields instead modify the risk-neutral drift and therefore affect the probabilities.

2.4 Exercise 3: check the impact of dividends on option prices

Consider the test case for European and American call and put options provided in the template. Create instances of your tree models with dividends and perform numerical tests.

Check the following: we know that an American call option and a European call option on a non-dividend paying stock have the same price. Introduce a large enough dividend to check that the American price and the European price of the call option no longer coincide.

2.5 Exercise 4: introduce a new product - Barrier options

Your boss tells you that the bank would like to start offering Barrier options to the clients. Barrier options are very popular in the context of foreign exchange rates and are a cheaper alternative to European calls and puts.

Barrier options pay/do not pay a put/call type payoff, depending on the fact that a barrier has been hit or not. If the barrier is hit, then an amount of cash called rebate can be paid either at maturity or at the day where the barrier is hit (rebate at hit). Here is a list of the payoffs.

1. **IN Barriers** In options are paid for today but first come into existence if the asset price S hits the barrier H before expiration. It is possible to include a prespecified cash rebate K , which is paid out at option expiration if the option has not been knocked in during its lifetime.

- (a) Down-and-in call $S > H$ Payoff: $\max(S - X; 0)$ if $S \leq H$ before T else K at expiration.

$$(S_T - X)^+ \mathbf{1}_{\{\min_{0 \leq t \leq T} S_t < H\}}$$

- (b) Up-and-in call $S < H$ Payoff: $\max(S - X; 0)$ if $S \geq H$ before T else K at expiration.

$$(S_T - X)^+ \mathbf{1}_{\{\max_{0 \leq t \leq T} S_t > H\}}$$

- (c) Down-and-in put $S > H$ Payoff: $\max(X - S; 0)$ if $S \leq H$ before T else K at expiration.

$$(X - S_T)^+ \mathbf{1}_{\{\min_{0 \leq t \leq T} S_t < H\}}$$

- (d) Up-and-in put $S < H$ Payoff: $\max(X - S; 0)$ if $S \geq H$ before T else K at expiration.

$$(X - S_T)^+ \mathbf{1}_{\left\{ \max_{0 \leq t \leq T} S_t > H \right\}}$$

2. **OUT Barriers** Out options are similar to standard options except that the option becomes worthless if the asset price S hits the barrier before expiration. It is possible to include a prespecified cash rebate K , which is paid out if the option is knocked out before expiration.

- Down-and-out call $S > H$ Payoff: $\max(S - X; 0)$ if $S > H$ before T else K at hit.

$$(S_T - X)^+ \mathbf{1}_{\{\min_{0 \leq t \leq T} S_t > H\}}$$

- Up-and-out call $S < H$ Payoff: $\max(S - X; 0)$ if $S < H$ before T else K at hit.

$$(S_T - X)^+ \mathbf{1}_{\{\max_{0 \leq t \leq T} S_t < H\}}$$

- Down-and-out put $S > H$ Payoff: $\max(X - S; 0)$ if $S > H$ before T else K at hit.

$$(X - S_T)^+ \mathbf{1}_{\{\min_{0 \leq t \leq T} S_t > H\}}$$

- Up-and-out put $S < H$ Payoff: $\max(X - S; 0)$ if $S < H$ before T else K at hit.

$$(X - S_T)^+ \mathbf{1}_{\{\max_{0 \leq t \leq T} S_t < H\}}$$

The task seems at first impossible, but then you realize that you do not need to implement 8 different options: since you already know how to price European options with trees you can use these IN-OUT-Identities

Down-and-in call

$$V_{\text{DI Call}} = V_{\text{European Call}} - V_{\text{DO Call}}$$

Up-and-in call

$$V_{\text{UI Call}} = V_{\text{European Call}} - V_{\text{UO Call}}$$

Down-and-in put

$$V_{\text{DIPut}} = V_{\text{European Put}} - V_{\text{DOPut}}$$

Up-and-in put

$$V_{\text{UIPut}} = V_{\text{EuropeanPut}} - V_{\text{UOPut}}$$

Equivalently, in a compact notation:

$$\text{Down-In} = \text{European} - \text{Down-Out}$$

$$\text{Up-In} = \text{European} - \text{Up-Out}$$

for both calls and puts. Also, a colleague of yours has implemented for you all the continuous time formulas for barrier options in the package `it.univr.fima.barrieroptionsformulas` in the Black Scholes case, so you will be able to compare your prices against the closed form formulas in the continuous time case. You do not need to study the formulas, anyway you can find them in the appendix.

A way to compute the prices can be for example

```

1  @Override
2  public RandomVariable[] getValues(final double evaluationTime, final TreeModel model)
3  {
4      validate(evaluationTime, model);
5
6      if(isInOption()) {
7          return getValuesViaInOutParity(evaluationTime, model);
8      }
9
10     return getValuesForOutOption(evaluationTime, model);

```

where you call two private methods with the signature

```

1  private RandomVariable[] getValuesViaInOutParity(final double evaluationTime, final
2  TreeModel model){
3
4      // do stuff here
5  }
6
7  private RandomVariable[] getValuesForOutOption(final double evaluationTime, final
8  TreeModel model) {
9
10     // do stuff here

```

2.6 Exercise 5: compare Barrier option prices produced by the trees against the closed form formulas

Once you have implemented your Barrier Option class, test the values you produce against those produced by the class `it.univr.fima.barrieroptionsformulas.BarrierOption`.

WARNING: in the finmath java library, under `net.finmath.functions` you can find another class with an implementation of the closed-form formulas for Barrier Options. Do not use it: There is a small typo in a formula which I am going to fix by using the file that comes with the project work template.

Chapter 3

Practical Aspects

3.1 General Rules

- Deadline for submitting the solution: 20 May 2026 at 12:00.
- Mark in case you do not submit the PW: 0/30 with a weight of 30% on the final grade.
- You have to produce:
 1. A small pdf of max 3 pages where you explain what you did and your results.
 2. A zip file with the whole **working** Eclipse project.
- Send me the project and the pdf via a OneDrive link or a file sharing service of your choice. DO NOT SEND FILES VIA EMAIL since the Antivirus software of the University will identify every java file as a potential Virus.
- On cheating: you can imagine how many hours of work go into preparing a well structured programming assignment that really teaches you something. Hence, please, do not waste my time. Do it seriously, do it professionally. We all know that it is too easy to cheat nowadays. Ask yourself if you really want to learn something for your professional future.

3.2 Project for working students

Students who contacted me via Email and received my authorization to work alone should limit themselves to implement in Exercise 1 a single dividend, adjust only Cox Ross Rubinstein and perform the check on American call options with dividends.

Appendix A

Impact of a non-constant dividend yield

You are not required to deal with this case in the exercises. This appendix is just to answer the natural question: if we decided to include non-constant dividend yields, what should we do?

Continuous dividend yield: $D(t) = \exp\left(\int_0^t q(s) ds\right), \quad Q_n = \int_{t_n}^{t_{n+1}} q(s) ds.$

Risk-neutral drift condition: $\mathbb{E}^{\mathbb{Q}}\left[\frac{S_{n+1}}{S_n} \middle| \mathcal{F}_{t_n}\right] = e^{r\Delta t - Q_n}.$

1. Cox–Ross–Rubinstein (CRR)

$$u = e^{\sigma\sqrt{\Delta t}}, \quad d = e^{-\sigma\sqrt{\Delta t}} = \frac{1}{u}.$$

$$p_u^{(n)} = \frac{e^{r\Delta t - Q_n} - d}{u - d}, \quad p_d^{(n)} = 1 - p_u^{(n)} = \frac{u - e^{r\Delta t - Q_n}}{u - d}.$$

2. Jarrow–Rudd

$$u_n = \exp\left(r\Delta t - Q_n - \frac{1}{2}\sigma^2\Delta t + \sigma\sqrt{\Delta t}\right),$$

$$d_n = \exp\left(r\Delta t - Q_n - \frac{1}{2}\sigma^2\Delta t - \sigma\sqrt{\Delta t}\right).$$

$$p_u = \frac{1}{2}, \quad p_d = \frac{1}{2}.$$

3. Boyle Trinomial

$$u = e^{\sigma\sqrt{2\Delta t}}, \quad m = 1, \quad d = e^{-\sigma\sqrt{2\Delta t}} = \frac{1}{u}.$$

$$R_n = e^{r\Delta t - Q_n}, \quad M_{2,n} = e^{2r\Delta t - 2Q_n + \sigma^2\Delta t}.$$

$$p_u^{(n)} = \frac{(R_n - 1)(d^2 - 1) - (M_{2,n} - 1)(d - 1)}{(u - 1)(d^2 - 1) - (d - 1)(u^2 - 1)},$$

$$p_d^{(n)} = \frac{(u - 1)(M_{2,n} - 1) - (u^2 - 1)(R_n - 1)}{(u - 1)(d^2 - 1) - (d - 1)(u^2 - 1)},$$

$$p_m^{(n)} = 1 - p_u^{(n)} - p_d^{(n)}.$$

Conclusion: if we do not restrict q to be constant then we need to recompute the growth factors and the risk neutral probabilities at every time step. The same would happen if we also decided to move to non-uniform time discretizations.

Appendix B

Recombining tree with dividends

A visual example of the effect of a `DiscreteProportionalDividends` with two different `DividendEvent`.

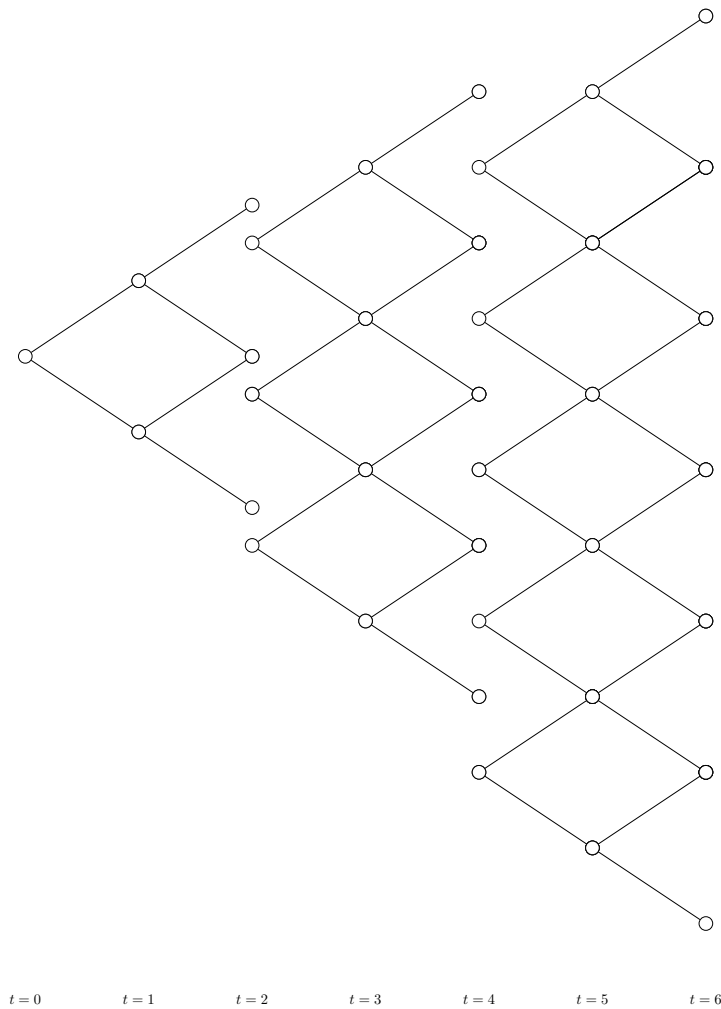


Figure B.1: Tree with 2 multiplicative dividend payments

Appendix C

The exact pricing formulas for Barrier Options under the Black-Scholes model

Please, you do not need to study them. I am reporting them here just for reference. These formulas were derived by Merton (1973) and Reiner and Rubinstein (1994). Define:

$$\begin{aligned}
A &= \phi S e^{(b-r)T} N(\phi x_1) - \phi X e^{-rT} N(\phi x_1 - \phi \sigma \sqrt{T}) \\
B &= \phi S e^{(b-r)T} N(\phi x_2) - \phi X e^{-rT} N(\phi x_2 - \phi \sigma \sqrt{T}) \\
C &= \phi S e^{(b-r)T} (H/S)^{2(\mu+1)} N(\eta y_1) - \phi X e^{-rT} (H/S)^{2\mu} N(\eta y_1 - \eta \sigma \sqrt{T}) \\
D &= \phi S e^{(b-r)T} (H/S)^{2(\mu+1)} N(\eta y_2) - \phi X e^{-rT} (H/S)^{2\mu} N(\eta y_2 - \eta \sigma \sqrt{T}) \\
E &= K e^{-rT} \left[N(\eta x_2 - \eta \sigma \sqrt{T}) - (H/S)^{2\mu} N(\eta y_2 - \eta \sigma \sqrt{T}) \right] \\
F &= K \left[(H/S)^{\mu+\lambda} N(\eta z) + (H/S)^{\mu-\lambda} N(\eta z - 2\eta \lambda \sigma \sqrt{T}) \right]
\end{aligned}$$

where

$$\begin{aligned}
x_1 &= \frac{\ln(S/X)}{\sigma \sqrt{T}} + (1 + \mu) \sigma \sqrt{T} & x_2 &= \frac{\ln(S/H)}{\sigma \sqrt{T}} + (1 + \mu) \sigma \sqrt{T} \\
y_1 &= \frac{\ln(H^2/(SX))}{\sigma \sqrt{T}} + (1 + \mu) \sigma \sqrt{T} & y_2 &= \frac{\ln(H/S)}{\sigma \sqrt{T}} + (1 + \mu) \sigma \sqrt{T} \\
z &= \frac{\ln(H/S)}{\sigma \sqrt{T}} + \lambda \sigma \sqrt{T} & \mu &= \frac{b - \sigma^2/2}{\sigma^2} & \lambda &= \sqrt{\mu^2 + \frac{2r}{\sigma^2}}
\end{aligned}$$

Prices of IN barriers

Down-and-in call $S > H$ Payoff: $\max(S - X; 0)$ if $S \leq H$ before T else K at expiration.

$$\begin{aligned}
c_{di}(X > H) &= C + E & \eta &= 1, \phi = 1 \\
c_{di}(X < H) &= A - B + D + E & \eta &= 1, \phi = 1
\end{aligned}$$

Up-and-in call $S < H$ Payoff: $\max(S - X; 0)$ if $S \geq H$ before T else K at expiration.

$$\begin{aligned} c_{ui}(X > H) &= A + E & \eta = -1, \phi = 1 \\ c_{ui}(X < H) &= B - C + D + E & \eta = -1, \phi = 1 \end{aligned}$$

Down-and-in put $S > H$ Payoff: $\max(X - S; 0)$ if $S \leq H$ before T else K at expiration.

$$\begin{aligned} p_{di}(X > H) &= B - C + D + E & \eta = 1, \phi = -1 \\ p_{di}(X < H) &= A + E & \eta = 1, \phi = -1 \end{aligned}$$

Up-and-in put $S < H$ Payoff: $\max(X - S; 0)$ if $S \geq H$ before T else K at expiration.

$$\begin{aligned} p_{ui}(X > H) &= A - B + D + E & \eta = -1, \phi = -1 \\ p_{ui}(X < H) &= C + E & \eta = -1, \phi = -1 \end{aligned}$$

Prices of OUT barriers

Down-and-in call $S > H$ Payoff: $\max(S - X; 0)$ if $S \leq H$ before T else K at expiration.

$$\begin{aligned} c_{di}(X > H) &= C + E & \eta = 1, \phi = 1 \\ c_{di}(X < H) &= A - B + D + E & \eta = 1, \phi = 1 \end{aligned}$$

Up-and-in call $S < H$ Payoff: $\max(S - X; 0)$ if $S \geq H$ before T else K at expiration.

$$\begin{aligned} c_{ui}(X > H) &= A + E & \eta = -1, \phi = 1 \\ c_{ui}(X < H) &= B - C + D + E & \eta = -1, \phi = 1 \end{aligned}$$

Down-and-in put $S > H$ Payoff: $\max(X - S; 0)$ if $S \leq H$ before T else K at expiration.

$$\begin{aligned} p_{di}(X > H) &= B - C + D + E & \eta = 1, \phi = -1 \\ p_{di}(X < H) &= A + E & \eta = 1, \phi = -1 \end{aligned}$$

Up-and-in put $S < H$ Payoff: $\max(X - S; 0)$ if $S \geq H$ before T else K at expiration.

$$\begin{aligned} p_{ui}(X > H) &= A - B + D + E & \eta = -1, \phi = -1 \\ p_{ui}(X < H) &= C + E & \eta = -1, \phi = -1 \end{aligned}$$